

# 2026-05-05-p1-f12-multi-provider-backend

## P1-F12 — Multi-Provider Backend Implementation Plan

**For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- [ ]) syntax for tracking.

**Goal:** Wire Anthropic / AWS Bedrock / Google Vertex AI / Azure OpenAI into a unified selection layer (--provider flag + HELIX\_LLM\_PROVIDER env + tview wizard + helixcode wizard cobra). Model lists exclusively from internal/verifier/. No new external deps. Non-breaking — Ollama remains the fallback.

**Architecture:** New internal/llm/{selector.go, wizard.go, wizard\_writer.go}. Extension to internal/llm/factory.go (NewCloudProvider). New cmd/cli/wizard\_cmd.go. cmd/cli/main.go startup replaces hardcoded Ollama with Selector.Resolve. The four cloud \*\_provider.go files already exist under internal/llm/ and are NOT rewritten.

**Tech Stack:** Go 1.26, testify v1.11, spf13/cobra v1.8, rivo/tview v0.42, gdamore/tcell/v2, aws-sdk-go-v2 + service/bedrockruntime, Azure azcore + azidentity, golang.org/x/oauth2/google. **No new external deps.**

**Spec:** docs/superpowers/specs/2026-05-05-p1-f12-multi-provider-backend-design.md (commit fd32a82)

**Working directory for go commands:** helix\_code/. Git from meta-repo root.

**Anti-bluff smoke (FULL 4-term):**

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|placeholder" \
    internal/llm/selector.go internal/llm/wizard.go internal/llm/
    wizard_writer.go \
    cmd/cli/wizard_cmd.go && echo BLUFF || echo clean
```

**Anti-bluff hot zone:** §5.2 of the spec — every PASS in this feature MUST distinguish LOCAL evidence from CLOUD evidence; cloud round-trips skip with SKIP-OK: P1-F12 cloud creds not provided when the corresponding env vars are absent. Bare skips are forbidden.

---

### Task list



P1-F12-T01 — bootstrap evidence + advance PROGRESS



P1-F12-T02 — provider.go: confirm unified Provider + LLMsVerifier integration (TDD audit, no rewrite)



P1-F12-T03 — `provider_anthropic.go`: confirm conformance to unified interface (TDD audit + gap fixes)



P1-F12-T04 — `provider_bedrock.go`: confirm conformance + verifier-backed `GetModels` (TDD)



P1-F12-T05 — `provider_vertex.go`: confirm conformance + verifier-backed `GetModels` (TDD)



P1-F12-T06 — `provider_azure.go`: confirm conformance + verifier-backed `GetModels` (TDD)



P1-F12-T07 — `provider_factory.go`: `NewCloudProvider` + selector precedence (TDD)



P1-F12-T08 — `wizard.go` + `wizard_writer.go`: `tview` TUI wizard (TDD with `tcell.SimulationScreen`)



P1-F12-T09 — `main.go` wiring + `helixcode` `wizard` cobra + integration test (gated, SKIP-OK on missing creds)



P1-F12-T10 — Challenge with runtime evidence (LOCAL always-runs, CLOUD credential-gated)



P1-F12-T11 — Feature 12 close-out + push 4 remotes

---

## Task 1: Bootstrap

Append F12 evidence section header (spec `fd32a82`), update PROGRESS current focus to F12, insert F12 task list (11 items) after F11's. Commit docs (P1-F12-T01): `bootstrap` Phase 1 / Feature 12 evidence + advance PROGRESS.

---

## Task 2: `provider.go` — audit + LLMsVerifier integration (TDD)

**Files:** `internal/llm/missing_types.go` (review-only), `internal/llm/verifier_integration.go` (review-only), new `internal/llm/provider_audit_test.go`.

The unified `Provider` interface and `VerifierModelSource` already exist. T02 produces an audit test that asserts the contract: every provider type returned by `factory.NewProvider` for `{anthropic, bedrock, vertexai, azure}` must implement `Provider` and yield a non-empty `GetModels()` slice when constructed with a `VerifierModelSource`-backed config.

Test:

```

func TestProviderInterface_AllFourCloudProvidersConform(t
    *testing.T) {
    types := []ProviderType{ProviderTypeAnthropic,
        ProviderTypeBedrock, ProviderTypeVertexAI,
        ProviderTypeAzure}
    for _, pt := range types {
        cfg := ProviderConfigEntry{Type: pt, APIKey: "test",
            Endpoint: "https://example.test"}
        p, err := NewProvider(cfg)
        require.NoError(t, err, "type %s", pt)
        assert.Equal(t, pt, p.GetType(), "type %s", pt)
        assert.NotEmpty(t, p.GetName(), "type %s", pt)
        assert.GreaterOrEqual(t, p.GetContextWindow(), 0, "type
            %s", pt)
    }
}

```

Subject: feat(P1-F12-T02): provider audit test confirms unified interface for 4 cloud types.

---

### Task 3: provider\_anthropic.go — conformance audit + gap fix (TDD)

**Files:** internal/llm/anthropic\_provider.go (existing, ~740 lines), internal/llm/anthropic\_provider\_test.go (existing).

Audit existing tests against the ProviderConfigEntry field map in spec §3.4. Gap-fix: ensure NewAnthropicProvider honours cfg.Endpoint as the ANTHROPIC\_BASE\_URL override and falls back to env. If a gap is found (verbatim), add a failing test, then fix the impl.

Test:

```

func TestAnthropicProvider_BaseURLPrecedence_ConfigOverEnv(t
    *testing.T) {
    t.Setenv("ANTHROPIC_BASE_URL", "https://env.example")
    cfg := ProviderConfigEntry{Type: ProviderTypeAnthropic,
        APIKey: "k", Endpoint: "https://cfg.example"}
    p, err := NewAnthropicProvider(cfg)
    require.NoError(t, err)
    // Use a probing http RoundTripper or unexported getter; if
    // unavailable, assert via test-only accessor
    assert.Contains(t, p.BaseURL(), "cfg.example")
}

```

Subject: feat(P1-F12-T03): anthropic provider audit + base URL precedence.

---

## Task 4: provider\_bedrock.go — conformance audit + verifier-backed GetModels (TDD)

**Files:** internal/llm/bedrock\_provider.go (existing, ~1086 lines), internal/llm/bedrock\_provider\_test.go.

Audit + gap-fix: BedrockProvider.GetModels MUST source from VerifierModelSource (filtered by ProviderType == bedrock) at construction. Hardcoded slices are forbidden by CONST-036/037.

Test:

```
func TestBedrockProvider_GetModels_SourcedFromVerifier(t
    *testing.T) {
    srv := startEmbeddedVerifier(t) // existing helper from
        internal/verifier
    adapter := verifier.NewAdapter(srv.URL())
    src := NewVerifierModelSource(adapter)
    cfg := ProviderConfigEntry{Type: ProviderTypeBedrock,
        Parameters: map[string]any{"region": "us-east-1"}}
    p, err := newBedrockProviderWithSource(cfg,
        src) // test-only ctor
    require.NoError(t, err)
    models := p.GetModels()
    require.NotEmpty(t, models)
    for _, m := range models {
        assert.Equal(t, ProviderTypeBedrock, m.Provider)
    }
}
```

Subject: feat(P1-F12-T04): bedrock GetModels routes through VerifierModelSource.

---

## Task 5: provider\_vertex.go — conformance audit + verifier-backed GetModels (TDD)

**Files:** internal/llm/vertexai\_provider.go (existing, ~1022 lines), internal/llm/vertexai\_provider\_test.go.

Same shape as T04 but for ProviderTypeVertexAI. Verify Parameters["project\_id"] and Parameters["location"] are honoured.

Test:

```
func TestVertexProvider_GetModels_SourcedFromVerifier(t
    *testing.T) {
    srv := startEmbeddedVerifier(t)
    adapter := verifier.NewAdapter(srv.URL())
    src := NewVerifierModelSource(adapter)
```

```

cfg := ProviderConfigEntry{Type: ProviderTypeVertexAI,
    Parameters: map[string]any{"project_id": "test",
        "location": "us-central1"}}
p, err := newVertexProviderWithSource(cfg, src)
require.NoError(t, err)
models := p.GetModels()
require.NotEmpty(t, models)
for _, m := range models { assert.Equal(t,
    ProviderTypeVertexAI, m.Provider) }
}

```

Subject: feat(P1-F12-T05): vertex GetModels routes through VerifierModelSource.

---

## Task 6: provider\_azure.go — conformance audit + verifier-backed GetModels (TDD)

**Files:** internal/llm/azure\_provider.go (existing, ~868 lines), internal/llm/azure\_provider\_test.go.

Same shape as T04. Verify Endpoint (Azure resource) and Parameters["api\_version"], Parameters["deployment"] are honoured. Confirm azidentity.NewClientSecretCredential path validates against the spec §3.4 mapping.

Subject: feat(P1-F12-T06): azure GetModels routes through VerifierModelSource.

---

## Task 7: provider\_factory.go — NewCloudProvider + Selector (TDD)

**Files:** internal/llm/factory.go (modify), internal/llm/selector.go (new), internal/llm/selector\_test.go (new), internal/llm/factory\_test.go (extend).

Tests:

```

func TestNewCloudProvider_AcceptsAllFour(t *testing.T) {
    for _, pt := range []ProviderType{ProviderTypeAnthropic,
        ProviderTypeBedrock, ProviderTypeVertexAI,
        ProviderTypeAzure} {
        _, err := NewCloudProvider(ProviderConfigEntry{Type: pt,
            APIKey: "k", Endpoint: "https://x.y"})
        assert.NoError(t, err, "type %s", pt)
    }
}
func TestNewCloudProvider_RejectsNonCloudType(t *testing.T) {

```

```

_, err := NewCloudProvider(ProviderConfigEntry{Type:
    ProviderTypeOllama})
require.Error(t, err)
assert.Contains(t, err.Error(), "not a cloud provider")
}
func TestSelector_FlagWinsOverEnv(t *testing.T) { /*
    flag=anthropic, env=bedrock => anthropic */ }
func TestSelector_EnvWinsOverConfig(t *testing.T) { /*
    env=vertex, config=azure => vertex */ }
func TestSelector_ConfigWinsOverWizard(t *testing.T) { /* config
    exists => no wizard launch */ }
func TestSelector_OllamaFallback_NonTTY(t *testing.T) { /*
    AllowWizard=false, no flag/env/config => Ollama */ }
func TestSelector_RejectsUnknownProvider(t *testing.T) { /*
    flag="bogus" => error */ }

```

Implementation skeleton:

```

func NewCloudProvider(cfg ProviderConfigEntry) (Provider, error)
{
    switch cfg.Type {
    case ProviderTypeAnthropic, ProviderTypeBedrock,
        ProviderTypeVertexAI, ProviderTypeAzure:
        return NewProvider(cfg)
    default:
        return nil, fmt.Errorf("provider type %q is not a cloud
            provider (expected anthropic|bedrock|vertexai|azure)",
                cfg.Type)
    }
}

type Selector struct{ opts SelectorOptions }
func NewSelector(opts SelectorOptions) *Selector { return
    &Selector{opts: opts} }
func (s *Selector) Resolve(ctx context.Context) (Provider,
    ProviderType, error) {
    typ, err := s.resolveType(ctx)
    if err != nil { return nil, "", err }
    cfg := s.buildConfig(typ)
    p, err := NewProvider(cfg)
    if err != nil { return nil, typ, fmt.Errorf("provider %s:
        %w", typ, err) }
    return p, typ, nil
}

```

Subject: feat(P1-F12-T07): NewCloudProvider + Selector with  
flag>env>config>wizard precedence.

---

## Task 8: wizard.go + wizard\_writer.go — tview TUI (TDD)

**Files:** internal/llm/wizard.go, internal/llm/wizard\_writer.go, internal/llm/wizard\_test.go, internal/llm/wizard\_writer\_test.go.

Tests use `tcell.NewSimulationScreen()` for headless rendering. Assertions: cell content matches expected glyphs at known coords; on-Save callback writes a config file at the configured path with mode 0600.

Tests:

```
func TestWizard_RendersAllFourProviderOptions(t *testing.T) {
    sim := tcell.NewSimulationScreen("UTF-8")
    require.NoError(t, sim.Init())
    defer sim.Fini()
    sim.SetSize(80, 24)
    w := NewWizard(&memWriter{}, nil)
    go func() { _, _ = w.RunHeadless(context.Background(), sim) }
    ()
    waitForRender(sim)
    cells, _, _ := sim.GetContents()
    text := cellsToString(cells, 80)
    for _, p := range []string{"anthropic", "bedrock", "vertex",
        "azure"} {
        assert.Contains(t, text, p)
    }
}

func TestWizard_DynamicModelDropdown_ViaVerifier(t *testing.T) {
    /* verifier-backed dropdown populated */
}
func TestWizard_VerifierOffline_FallbackBannerShown(t
    *testing.T) { /* yellow warn banner present */ }
func TestWizard_ValidatesMandatoryFieldsPerProvider(t
    *testing.T) { /* per §3.4 */ }
func TestWizard_CancelReturnsError_NoFileWritten(t *testing.T) {
    /* ErrWizardCancelled, file absent */ }
func TestWizardWriter_File_0_EXCL_PreventsOverwrite(t
    *testing.T) {
    dir := t.TempDir()
    w := &FileWizardConfigWriter{path: filepath.Join(dir,
        "config.yaml")}
    require.NoError(t, w.Write(WizardForm{Provider:
        ProviderTypeAnthropic, APIKey: "k"}))
    err := w.Write(WizardForm{Provider: ProviderTypeAnthropic,
        APIKey: "k2"})
    require.ErrorIs(t, err, os.ErrExist)
}
func TestWizardWriter_File_Mode0600(t *testing.T) {
    dir := t.TempDir()
```

```

w := &FileWizardConfigWriter{path: filepath.Join(dir,
    "config.yaml")}
require.NoError(t, w.Write(WizardForm{Provider:
    ProviderTypeAnthropic, APIKey: "k"}))
info, err := os.Stat(w.path)
require.NoError(t, err)
assert.Equal(t, os.FileMode(0600), info.Mode().Perm())
}

```

Subject: feat(P1-F12-T08): tview wizard + WizardConfigWriter (mode 0600, O\_EXCL).

---

## Task 9: main.go wiring + helixcode wizard cobra + integration test

**Files:** cmd/cli/main.go (modify), cmd/cli/wizard\_cmd.go (new), cmd/cli/wizard\_cmd\_test.go (new), tests/integration/multi\_provider\_test.go (new, //go:build integration).

Replace the hardcoded `llm.NewOllamaProvider(...)` in `NewCLI()` with:

```

sel := llm.NewSelector(llm.SelectorOptions{
    Flag:      flagProvider,
    Env:       os.Getenv("HELIX_LLM_PROVIDER"),
    ConfigPath: filepath.Join(userConfigDir, "helixcode",
        "config.yaml"),
    AllowWizard: term.IsTerminal(int(os.Stdin.Fd())),
    Verifier:    verifierAdapter,
})
provider, _, err := sel.Resolve(ctx)

```

Add `helixcode wizard cobra` subcommand mirroring F09/F10/F11 patterns; it always launches `Wizard.Run` (overwrites prior config after y/N confirm).

Integration test:

```

//go:build integration
// +build integration

```

```

func TestMultiProvider_Anthropic_Health(t *testing.T) {
    if os.Getenv("ANTHROPIC_API_KEY") == "" {
        t.Skip("SKIP-OK: P1-F12 cloud creds not provided")
    }
    cfg := llm.ProviderConfigEntry{Type:
        llm.ProviderTypeAnthropic, APIKey:
        os.Getenv("ANTHROPIC_API_KEY")}
    p, err := llm.NewCloudProvider(cfg)
    require.NoError(t, err)
    health, err := p.GetHealth(context.Background())
}

```



```

    require.NoError(t, err)
    assert.Equal(t, "healthy", health.Status)
}
// + TestMultiProvider_Bedrock_Health,
//   TestMultiProvider_Vertex_Health,
//   TestMultiProvider_Azure_Health
// + TestMultiProvider_VerifierBackedModelList (always runs, no
//   creds needed)

```

Subject: feat(P1-F12-T09): wire selector into main.go + helixcode wizard cobra + integration tests.

---

## Task 10: Challenge with runtime evidence

**Files:** tests/integration/cmd/p1f12\_challenge/main.go (new), challenges/p1-f12-multi-provider/CHALLENGE.md (new), challenges/p1-f12-multi-provider/run.sh (new).

The harness **MUST** print two distinct sections:

```

=== LOCAL EVIDENCE (always runs) ===
[PASS] factory: anthropic constructed
[PASS] factory: bedrock constructed
[PASS] factory: vertexai constructed
[PASS] factory: azure constructed
[PASS] factory: rejects ollama as non-cloud
[PASS] selector: --provider flag wins over env
[PASS] selector: env wins over config
[PASS] selector: Ollama fallback when nothing set + non-TTY
[PASS] wizard: renders 4 provider options (tcell.SimulationScreen)
[PASS] wizard: model dropdown populated from VerifierModelSource (embedded)
[PASS] wizard: file write mode 0600 + 0_EXCL

=== CLOUD EVIDENCE (credential-gated) ===
[SKIP] anthropic Health: SKIP-OK: P1-F12 cloud creds not provided
[SKIP] bedrock Health:   SKIP-OK: P1-F12 cloud creds not provided
[SKIP] vertex Health:    SKIP-OK: P1-F12 cloud creds not provided
[SKIP] azure Health:     SKIP-OK: P1-F12 cloud creds not provided

```

SUMMARY: LOCAL=11/11 PASS; CLOUD=0/4 (SKIPPED, creds missing)

When env vars are present, the corresponding line flips from [SKIP] to [PASS] with real Health(ctx) round-trip evidence (status + latency).

Anti-bluff smoke clean check appended. Dual commit (Challenges submodule + meta-repo bump). Verbatim evidence in 06\_phase\_1\_evidence.md.

Subject: feat(P1-F12-T10): challenge with runtime evidence (LOCAL + credential-gated CLOUD).

---

## Task 11: Close-out + push

Tick 11 items in PROGRESS, advance PROGRESS focus to F13 candidate, run final verification (make verify-compile, anti-bluff smoke), commit chore(P1-F12-T11): close out feature 12 — multi-provider backend, push 4 remotes non-force.

---

### Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 interface audit, T03–T06 per-provider audits, T07 factory + selector, T08 wizard, T09 main wiring + integration tests, T10 Challenge, T11 close-out.
2. **TDD:** every code task starts with failing tests; per-provider audits add tests only when a gap is identified.
3. **Type consistency:** Selector, SelectorOptions, WizardForm, Wizard, WizardConfigWriter, FileWizardConfigWriter, NewCloudProvider consistent across spec and plan.
4. **Cross-platform:** pure Go + cloud SDKs already in go.mod; tview headless via tcell.SimulationScreen — no new deps.
5. **Anti-bluff:** full 4-term smoke + Challenge with explicit LOCAL/CLOUD separation + SKIP-OK: P1-F12 cloud creds not provided markers per Universal Rule.
6. **No new deps:** verified — aws-sdk-go-v2/service/bedrockruntime, Azure/azure-sdk-for-go/sdk/{azcore,azidentity}, golang.org/x/oauth2/google, rivo/tview, gdamore/tcell/v2 all present in helix\_code/go.mod.
7. **Branch + push:** stays on main, non-force to all four remotes (per CONST-043).
8. **Reality check:** 4 cloud \*\_provider.go files (anthropic/bedrock/azure/vertexai) ALREADY exist in internal/llm/. F12 is the selection/UX/wizard/factory layer plus per-provider verifier-routing audits — NOT a from-scratch implementation. The porting doc's brand-new Chat/StreamChat interface is intentionally NOT adopted; doing so would force a rewrite of every existing provider and contradict the non-breaking mandate (Q3=B).